

LEZIONE 24

RDD

&&

ANALISI OBJECT ORIENTED

CON BCE

Ingegneria del Software e Progettazione Web
Università degli Studi di Tor Vergata - Roma

Guglielmo De Angelis
guglielmo.deangelis@iasi.cnr.it

RDD

- un modo di pensare alla progettazione O.O.
 - **R**esponsibility **D**riven **D**esign
- le classi e/o le istanze in un sistema software sono associati ad una descrizione delle loro responsabilità
 - sono assegnate dal progettista
 - obblighi sulla struttura o sul comportamento in relazione al suo ruolo all'interno del sistema
 - obblighi sulla struttura o sul comportamento
 - responsabilità di fare
 - responsabilità di conoscere
 - responsabilità di collaborare

... uno scenario tipico:

- come individuare le classi di analisi?
- esistono tecniche a supporto di questa attività?
- le classe di analisi hanno tutte/sempre la stessa funzione?

... uno scenario tipico:

- come individuare le classi di analisi?
- esistono tecniche a supporto di questa attività? **SI!**
- le classe di analisi hanno tutte/sempre la stessa funzione? **Ovviamente NO!**

... uno scenario tipico:

- come individuare le classi di analisi?
- esistono tecniche a supporto di questa attività? **SI!**
- le classe di analisi hanno tutte/sempre la stessa funzione? **Ovviamente NO!**

RUP, ad esempio, suggerisce agli analisti di distinguere le classi di analisi in 3 macro categorie

- **boundary**
- **control**
- **entity**

PARENTESI : perché si ha bisogno di modellare aspetti non previsti dal linguaggio ?!?!

- principali motivazioni :
 - il dominio di applicazione o il committente hanno bisogno di esprimere concetti (sufficientemente) generali che non sono nativamente inclusi nel linguaggio di modellazione (i.e. UML)
 - il modellista ha bisogno di rappresentare aspetti legati allo specifico processo di sviluppo adottato
 - dando differenti interpretazioni ad uno stesso elemento del linguaggio in base alle fasi del processo
 - il modellista vuole raffinare i modelli esistenti in funzione del contesto
 - risolvendo eventuali ambiguità
 - per una generazione del codice più efficiente/efficace

PARENTESI : stereotipi

- gli stereotipi sono il principale meccanismo per la personalizzazione di UML
- uno stereotipo estende il significato di un elemento del linguaggio (meta-classe)
 - il risultato è un nuovo elemento nel linguaggio
 - l'elemento del linguaggio dove è possibile applicare uno stereotipo è chiamato “base class”
- attraverso la definizione di uno stereotipo possono essere definiti
 - ulteriori interpretazioni per un elemento
 - ulteriori caratteristiche di un elemento
 - ulteriori relazioni con altri elementi

BCE : quali responsabilità?

- **boundary**: mediano l'interazione tra il sistema e
 - rappresentano elementi al confine del sistema
 - regolano l'interazione con gli altri sistemi
- **control**: coordinano le attività del sistema
 - rappresentano il controllo del sistema (i.e. il controllo del sistema)
- **entity**: rappresentano le entità del sistema
 - sono le entità del sistema, cioè trasposizioni logiche del mondo reale
 - sono le entità del sistema, cioè trasposizioni logiche del mondo reale
 - dipendono dal problema
 - sono le entità del sistema, cioè trasposizioni logiche del mondo reale
- **model**: rappresentano il comportamento di una entità di dominio incapsulando un insieme coeso di dati
 - (solo in prima approssimazione,) definiscono i tipi di dati che vengono manipolati dall'insieme dei controller ed i cui valori sono esposti/richiesti dalle boundary

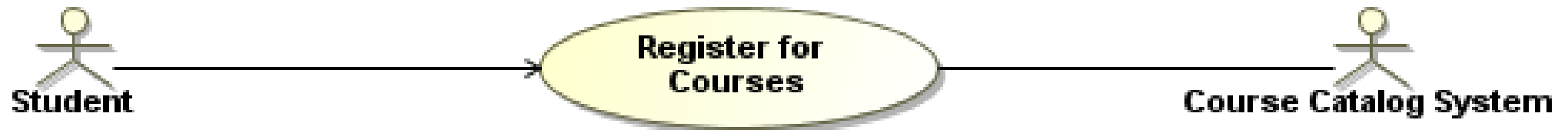
definire e marcare le classi di analisi utilizzando gli stereotipi

- <<boundary>>
- <<control>>
- <<entity>>

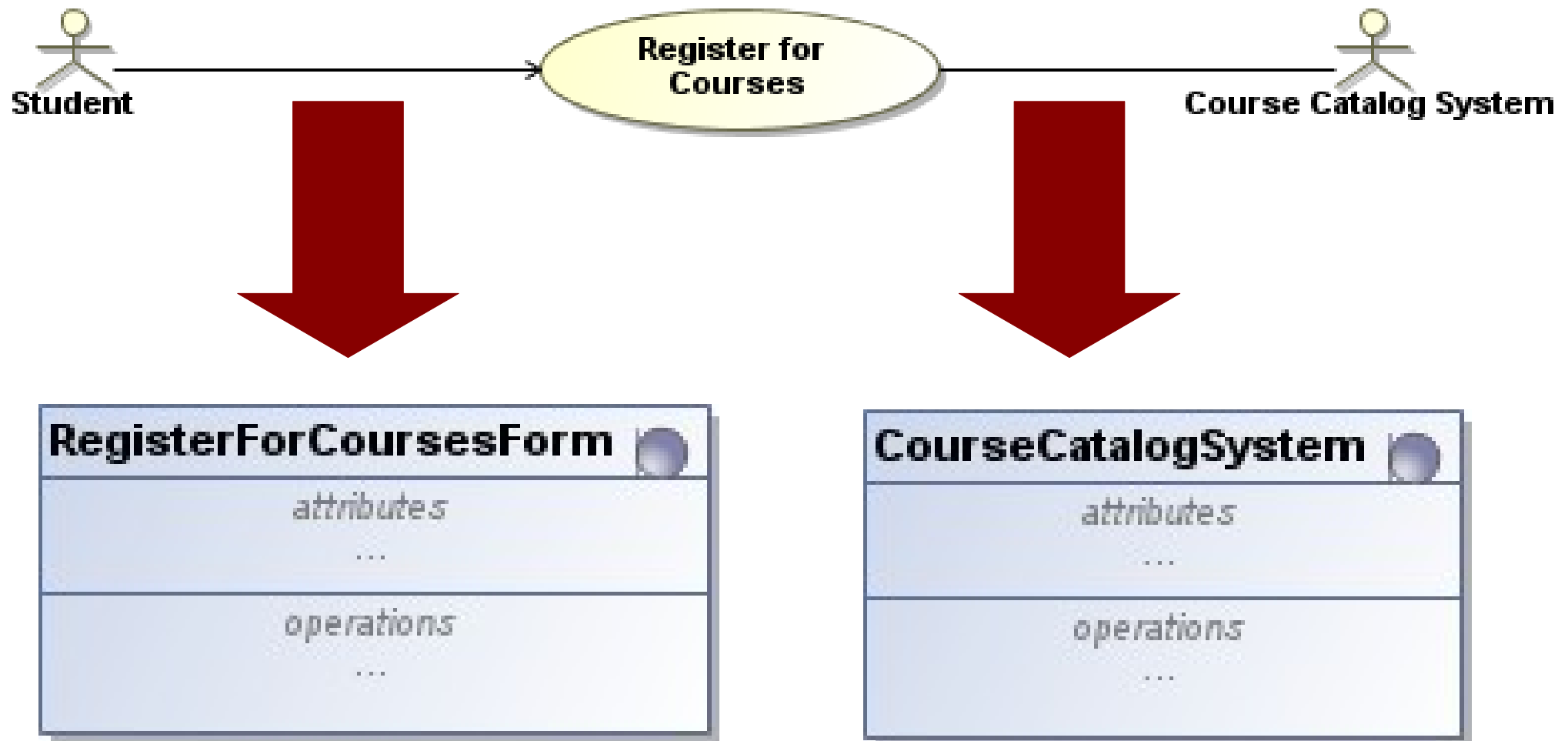
BCE : quali responsabilità?

- **boundary**: mediano l'interazione tra il sistema e l'ambiente
 - rappresentano elementi al confine del sistema
 - regolano l'interazione con gli attori
- **control**: coordinano il comportamento durante un caso d'uso del sistema
 - rappresentano comportamenti dipendenti dall'interazione attesa con il sistema (i.e. comportamento descritto dal caso d'uso)
 - sono indipendenti dal modo di attivazione dell'interazione
 - queste classi dovrebbero scaturire dall'analisi dominio del problema
- **entity**: rappresentano le astrazioni chiave del sistema, cioè trasposizioni logiche degli elementi reali del dominio considerato
 - sono indipendenti dall'ambiente di esecuzione
 - dipendono dall'analisi dominio del problema
 - glossario, use case, business model, etc.
 - modellano il comportamento di una entità di dominio incapsulando un insieme coeso di dati
 - (solo in prima approssimazione,) definiscono i tipi di dati che vengono manipolati dall'insieme dei controller ed i cui valori sono esposti/richiesti dalle boundary

BCE – boundary



BCE – boundary



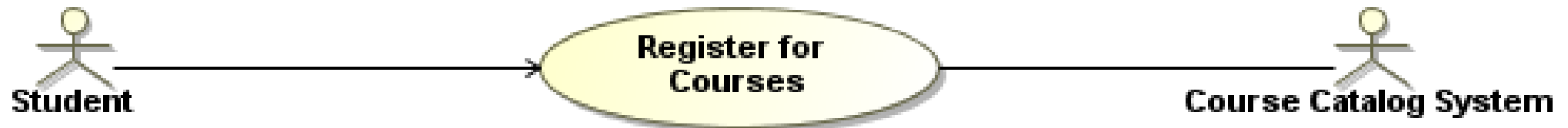
BCE – boundary



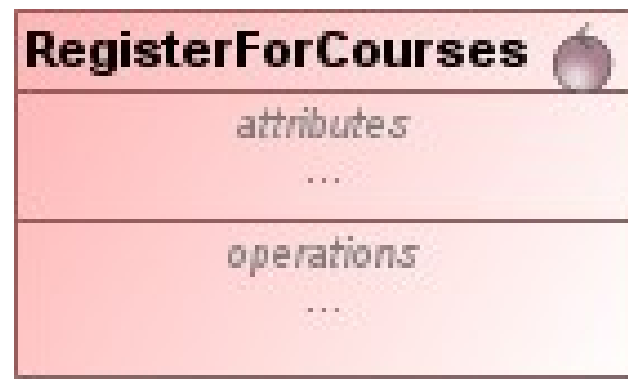
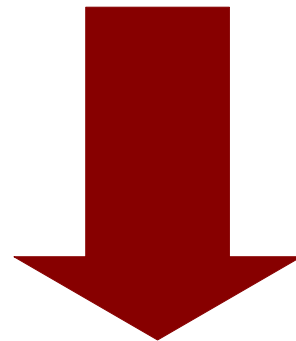
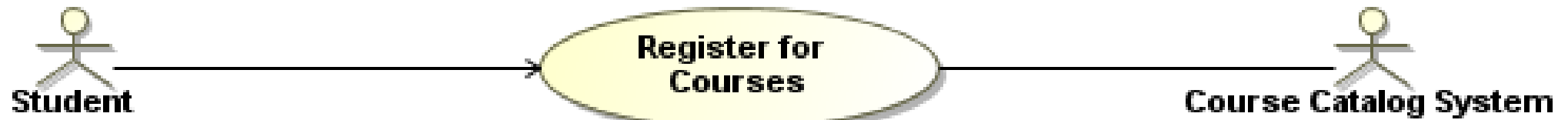
BCE : quali responsabilità?

- **boundary**: mediano l'interazione tra il sistema e l'ambiente
 - rappresentano elementi al confine del sistema
 - regolano l'interazione con gli attori
- **control**: coordinano il comportamento durante un caso d'uso del sistema
 - rappresentano comportamenti dipendenti dall'interazione attesa con il sistema (i.e. comportamento descritto dal caso d'uso)
 - sono indipendenti dal modo di attivazione dell'interazione
 - queste classi dovrebbero scaturire dall'analisi dominio del problema
- **entity**: rappresentano le astrazioni chiave del sistema, cioè trasposizioni logiche degli elementi reali del dominio considerato
 - sono indipendenti dall'ambiente di esecuzione
 - dipendono dall'analisi dominio del problema
 - glossario, use case, business model, etc.
 - modellano il comportamento di una entità di dominio incapsulando un insieme coeso di dati
 - (solo in prima approssimazione,) definiscono i tipi di dati che vengono manipolati dall'insieme dei controller ed i cui valori sono esposti/richiesti dalle boundary

BCE – control



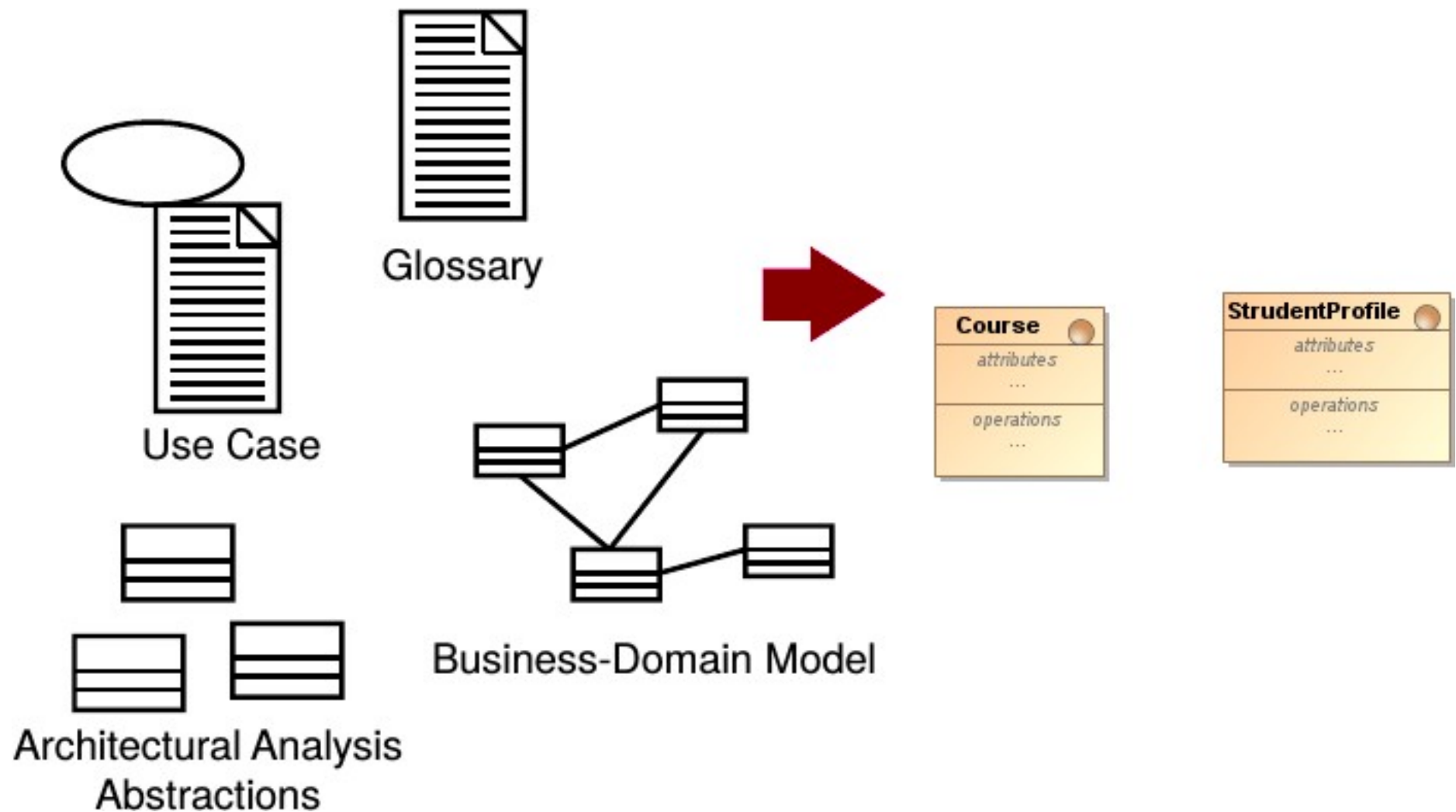
BCE – control



BCE : quali responsabilità?

- **boundary**: mediano l'interazione tra il sistema e l'ambiente
 - rappresentano elementi al confine del sistema
 - regolano l'interazione con gli attori
- **control**: coordinano il comportamento durante un caso d'uso del sistema
 - rappresentano comportamenti dipendenti dall'interazione attesa con il sistema (i.e. comportamento descritto dal caso d'uso)
 - sono indipendenti dal modo di attivazione dell'interazione
 - queste classi dovrebbero scaturire dall'analisi dominio del problema
- **entity**: rappresentano le astrazioni chiave del sistema, cioè trasposizioni logiche degli elementi reali del dominio considerato
 - sono indipendenti dall'ambiente di esecuzione
 - dipendono dall'analisi dominio del problema
 - glossario, use case, business model, etc.
 - modellano il comportamento di una entità di dominio incapsulando un insieme coeso di dati
 - (solo in prima approssimazione,) definiscono i tipi di dati che vengono manipolati dall'insieme dei controller ed i cui valori sono esposti/richiesti dalle boundary

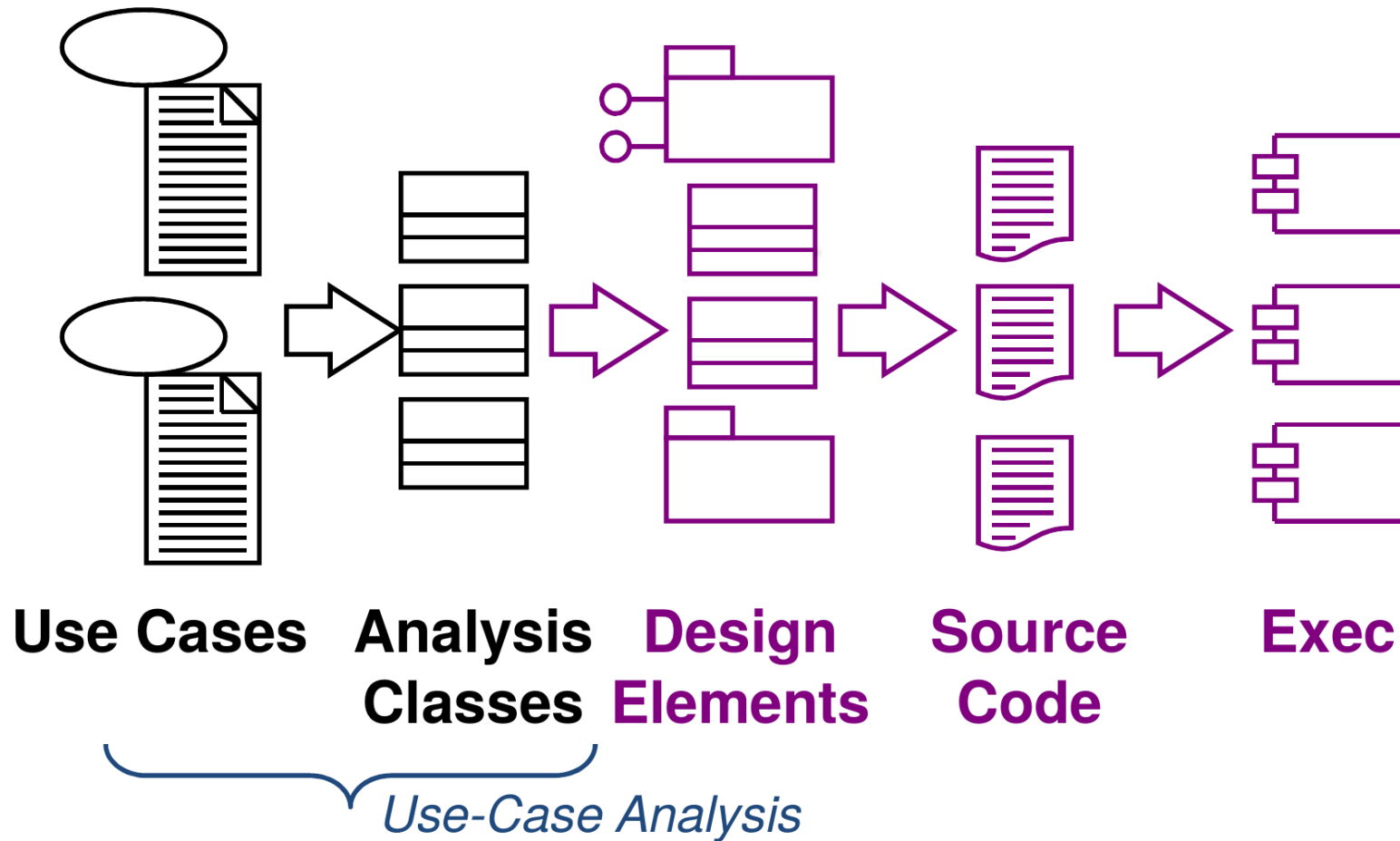
BCE – entity



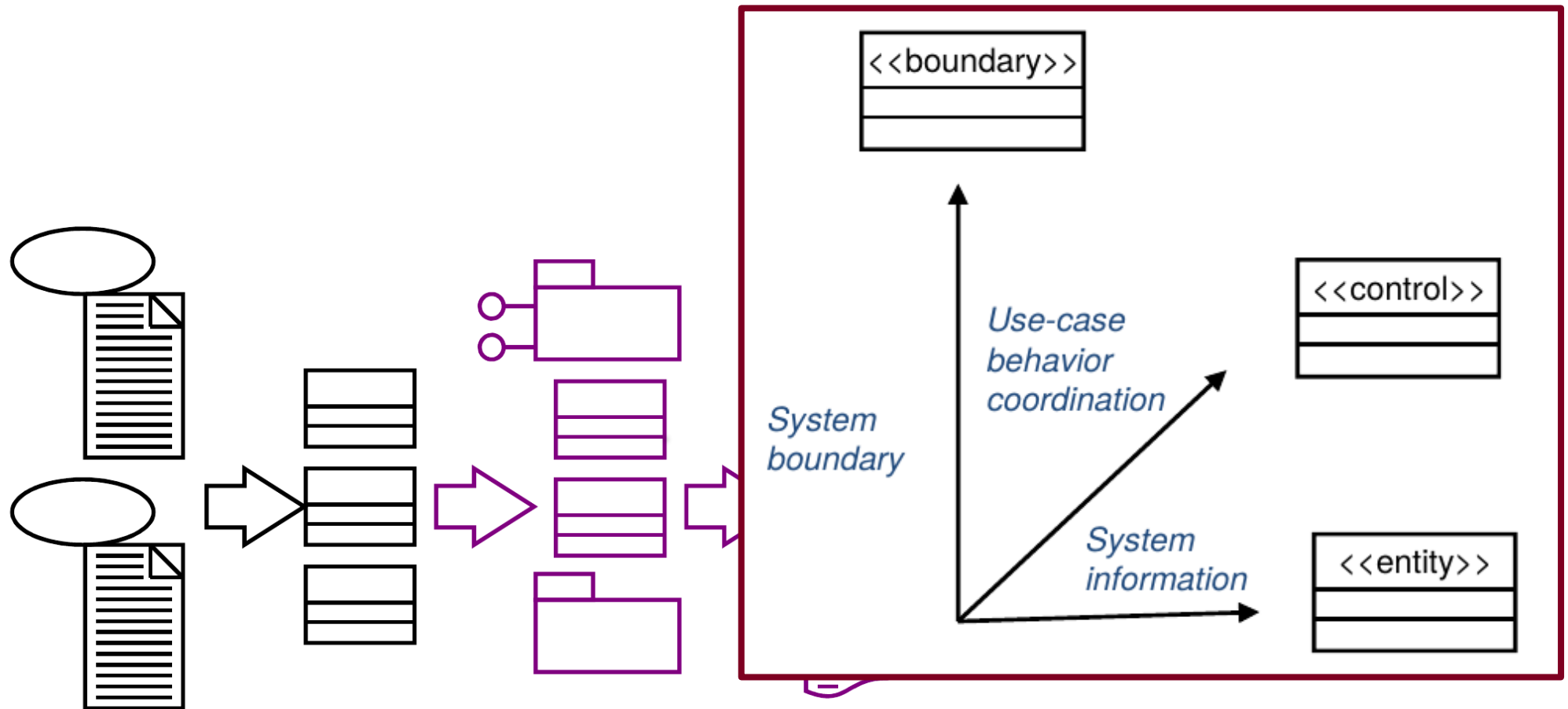
BCE : quali responsabilità?

- **boundary**: mediano l'interazione tra il sistema e l'ambiente
 - rappresentano elementi al confine del sistema
 - regolano l'interazione con gli attori
- **control**: coordinano il comportamento durante un caso d'uso del sistema
 - rappresentano comportamenti dipendenti dall'interazione attesa con il sistema (i.e. comportamento descritto dal caso d'uso)
 - sono indipendenti dal modo di attivazione dell'interazione
 - queste classi dovrebbero scaturire dall'analisi dominio del problema
- **entity**: rappresentano le astrazioni chiave del sistema, cioè trasposizioni logiche degli elementi reali del dominio considerato
 - sono indipendenti dall'ambiente di esecuzione
 - dipendono dall'analisi dominio del problema
 - glossario, use case, business model, etc.
 - modellano il comportamento di una entità di dominio incapsulando un insieme coeso di dati
 - (solo in prima approssimazione,) definiscono i tipi di dati che vengono manipolati dall'insieme dei controller ed i cui valori sono esposti/richiesti dalle boundary

artefatti SW e BCE



artefatti SW e BCE



Use Cases

**Analysis
Classes**

**Design
Elements**

**Source
Code**

Exec

Use-Case Analysis

... in conclusione

- il pattern di analisi BCE consente di affrontare il seguente schema di problema
 - Quali classi partecipano ad un caso d'uso?
 - Quante sono le classi di analisi per un caso d'uso?
- lo schema di soluzione proposto viene identificato con il termine **VOPC**: View of Participating Classes
 - vista della classi partecipanti ad ogni caso d'uso

bibliografia di riferimento

- “Design Patterns – Elementi per il riuso di software a oggetti”, Gamma, Helm, Johnson, Vlissides (GoF). Addison-Wesley (1995).
 - Design Patterns: Elements of Reusable Object-Oriented Software
- “Applicare UML e i Pattern – Analisi e progettazione orientata agli oggetti”, Larman. 3za Edizione. Pearson (2005).
 - “Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development”